BOOTCAMP DATA SCIENCE · GRUPO 4

Hundir *la Flota*

Grupo 4
Entrega: 17 abr

Python 3.x | NumPy | 3 integrantes | 3 semanas · 30 mar – 17 abr

EQUIPO

SCRUM MASTER
Claudia

variables.py · main.py · README.md

- Constantes: BOARD_SIZE, SHIPS, símbolos
- Mensajes del juego
- Bucle while True en main.py
- Integración final de módulos
- Documentación README

DEVELOPER 1
Martha

clases.py

- Clase Tablero — constructor __init__
- place_ships() — colocación aleatoria
- receive_shot(row, col) → bool
- display(oculto) — impresión en consola
- 2 arrays numpy 10×10 (propio + tracking)

DEVELOPER 2
Isaac

funciones.py

- pedir_coordenadas() — validación input
- disparo_maquina() — coordenada aleatoria
- verificar_fin(tablero) → bool
- Evitar repetición de disparos
- Gestión de turnos y conteo

CRONOGRAMA — 3 SEMANAS · 30 MAR – 17 ABR

Semana 1 — Arranque

Lun 30 mar · Vie 4 abr

LUN 30 MAR · TODO EL EQUIPO
Kick-off: leer instrucciones, repartir tareas y crear la carpeta del proyecto

LUN 30 MAR · CLAUDIA
Escribir variables.py completo y compartirlo con Martha e Isaac — es bloqueante para ambos

MAR 31 MAR · CLAUDIA
Crear esqueleto de main.py con imports, bienvenida y estructura del bucle while

MAR 31 MAR · MARTHA
Diseñar la clase Tablero: constructor __init__ y los dos arrays numpy 10×10

MAR 31 MAR · ISAAC
Implementar pedir_coordenadas() con validación de rango y entrada inválida

MIÉ 2 ABR · MARTHA
Implementar place_ships() — colocación fija primero, luego aleatoria

MIÉ 2 ABR · ISAAC
Implementar disparo_maquina() — coordenada aleatoria sin repetición

VIE 4 ABR · TODO EL EQUIPO
Mini revisión: mostrar avances y sincronizar nombres de funciones entre módulos

Semana 2 — Desarrollo

Lun 7 abr · Vie 11 abr

LUN 7 ABR · MARTHA
Implementar receive_shot(row, col) → marcar impacto o agua en los arrays

MAR 8 ABR · MARTHA
Implementar display(oculto) — imprimir tablero propio y tablero enemigo

LUN 7 ABR · ISAAC
Implementar verificar_fin(tablero) — comprobar si ship_cells_remaining == 0

MAR 8 ABR · ISAAC
Hacer prueba unitaria de las tres funciones con casos límite

MIÉ 9 ABR · CLAUDIA
Integrar clases.py + funciones.py en main.py — primera partida funcional

JUE 10 ABR · CLAUDIA
Avanzar README.md — instrucciones, simbología y tabla de flota

VIE 11 ABR · TODO EL EQUIPO
Jugar una partida completa de prueba — del mensaje de bienvenida hasta victoria o derrota

Semana 3 — Entrega

Lun 14 abr · Jue 17 abr

⚓ ENTREGA: 17 ABR

LUN 14 ABR · TODO EL EQUIPO
Sesión de bug fixing: corregir disparos repetidos, tablero fuera de rango y fin de partida

LUN 14 ABR · MARTHA
Implementar colocación aleatoria de barcos si aún es fija — mejora opcional

MAR 15 ABR · ISAAC
Revisar que la máquina nunca repita coordenada y que los turnos alternen correctamente

MAR 15 ABR · CLAUDIA
Finalizar README — añadir instrucciones de instalación (pip install numpy) y cómo ejecutar

MIÉ 16 ABR · TODO EL EQUIPO
Hacer revisión final del código — limpiar, comentar y verificar todos los archivos

JUE 17 ABR · CLAUDIA
✅ Entregar el proyecto final — subir todos los archivos y el README al repositorio

DEPENDENCIAS ENTRE ARCHIVOS

variables.py

CLAUDIA · ENTREGAR LUN 30 MAR

No importa nada.
⚠ Todos dependen de este.

clases.py

MARTHA · ENTREGAR MIÉ 9 ABR

Importa: variables.py
numpy

funciones.py

ISAAC · ENTREGAR MIÉ 9 ABR

Importa: variables.py
random

main.py

CLAUDIA · INTEGRACIÓN MIÉ 9 ABR

Importa: variables.py
clases.py · funciones.py

SIMBOLOGÍA DEL TABLERO

~	O	X	*
Agua sin disparar	Barco intacto	Impacto en barco	Disparo fallado

CRITERIOS DE FINALIZACIÓN

1. **Entregar variables.py el lunes 30 de marzo** — sin esto Martha e Isaac no pueden empezar.
2. **Verificar que clases.py funciona de forma aislada** — poder crear un Tablero, colocar barcos e imprimir.
3. **Comprobar que funciones.py valida entradas** — rechazar coordenadas fuera de rango y disparos repetidos.
4. **Jugar una partida completa** — desde la bienvenida hasta el mensaje de victoria o derrota.
5. **Garantizar que no se repiten disparos** — ni del jugador ni de la máquina en ningún caso.
6. **Completar el README** — cualquier persona debe poder leerlo y ejecutar el juego sin instrucciones extra.